

C Programming Practice Set for Digital Design / ASIC Interviews

Bitwise operations • Data representation • Endianness • Pointers & registers • volatile/const
• Structs, unions & padding • Output-tracing traps

Every question is followed by a worked solution. Attempt each question on paper first (no compiler!), then check. Hardware interviewers care about your *reasoning at the bit level*, not memorized answers, so trace values in binary as you go.

How to use this set

Sections 1–3 are the heart of ASIC/RTL interview C. Sections 4–6 mirror the output-tracing style of your course quizzes but with hardware flavor. Section 7 is rapid-fire conceptual questions often asked verbally.

Section 1 — Bitwise Operations & Bit Manipulation (most important)

Convention: bit 0 is the least-significant bit (LSB). Assume 32-bit unsigned `int` unless stated. These are the single most frequently asked C questions in digital-design interviews because they map directly to masking, muxing and register manipulation in RTL.

Q1. Write one-line expressions to SET, CLEAR, TOGGLE and TEST the n-th bit of an unsigned int x.

Solution

```
x |= (1U << n);          /* SET   bit n : OR with a 1 in that position */
x &= ~(1U << n);         /* CLEAR bit n : AND with a 0 in that position */
x ^= (1U << n);          /* TOGGLE bit n: XOR with 1 flips the bit    */
bit = (x >> n) & 1U;     /* TEST  bit n : shift down, mask the LSB    */
```

Why 1U and not 1? If $n = 31$, shifting a signed 1 into the sign bit is undefined behaviour. Interviewers love this follow-up.

Q2. Write a function to check whether an unsigned integer is a power of 2. (Extremely common.)

Solution

```
int is_pow2(unsigned x)
{
    return x != 0 && (x & (x - 1)) == 0;
}
```

Key insight: a power of 2 has exactly one set bit, e.g. $8 = 1000$. Subtracting 1 flips all bits up to and including that bit (0111), so ANDing gives 0. The $x \neq 0$ guard is essential — $0 \& -1$ is also 0 but 0 is not a power of 2. Forgetting the guard is the classic mistake.

Q3. Count the number of set bits (popcount) in an unsigned int. Give the naive loop AND the Brian Kernighan trick, and state why the second is faster.

Solution

```
/* Naive: 32 iterations always */
int count1(unsigned x){ int c=0; while(x){ c += x & 1; x >>= 1; } return c; }

/* Kernighan: one iteration PER SET BIT */
int count2(unsigned x){ int c=0; while(x){ x &= (x - 1); c++; } return c; }
```

$x \& (x-1)$ clears the lowest set bit each pass, so 0x80000001 takes only 2 iterations instead of 32. In hardware terms this is the same identity used for one-hot checks.

Q4. Swap two integers without using a temporary variable. Mention the caveat.

Solution

```
a ^= b;
b ^= a;    /* b = b ^ (a^b) = a */
a ^= b;    /* a = (a^b) ^ a = b */
```

Caveat: if `a` and `b` are the same memory location (e.g. `swap(&x,&x)` through pointers), the value is destroyed to 0 after the first XOR. Also, on modern CPUs a temp variable is usually just as fast — say this to show judgment.

Q5. Reverse the bits of a 32-bit unsigned integer (bit 0 becomes bit 31, etc.). Give the simple loop version and the divide-and-conquer (mask-shift) version.

Solution

```
/* Simple: build result bit by bit */
```

```

unsigned rev(unsigned x)
{
    unsigned r = 0;
    for (int i = 0; i < 32; i++) {
        r = (r << 1) | (x & 1);
        x >>= 1;
    }
    return r;
}

/* Divide & conquer: swap halves, then quarters, ... O(log n) */
unsigned rev2(unsigned x)
{
    x = ((x & 0x55555555) << 1) | ((x >> 1) & 0x55555555); /* swap odd/even bits */
    x = ((x & 0x33333333) << 2) | ((x >> 2) & 0x33333333); /* swap bit pairs */
    x = ((x & 0x0F0F0F0F) << 4) | ((x >> 4) & 0x0F0F0F0F); /* swap nibbles */
    x = ((x & 0x00FF00FF) << 8) | ((x >> 8) & 0x00FF00FF); /* swap bytes */
    x = (x << 16) | (x >> 16); /* swap half-words */
    return x;
}

```

The mask-shift version is exactly how a hardware bit-reversal network is built — mentioning that scores points in an ASIC interview.

Q6. A 32-bit register REG has a 4-bit field MODE occupying bits [7:4]. Write C code to (a) read the field and (b) write the value m into the field without disturbing other bits.

Solution

```

#define MODE_SHIFT 4
#define MODE_MASK (0xFU << MODE_SHIFT) /* 0x000000F0 */

unsigned mode = (REG & MODE_MASK) >> MODE_SHIFT; /* (a) extract */

REG = (REG & ~MODE_MASK) | ((m << MODE_SHIFT) & MODE_MASK); /* (b) insert */

```

This read-modify-write pattern is THE canonical register-access idiom. Clearing the field first (& ~MASK) before ORing in the new value is what interviewers check; masking m protects against out-of-range values.

Q7. Without using * or /, multiply an integer by 8, divide by 4, and compute x*10 using shifts and adds.

Solution

```

y = x << 3; /* x * 8 */
y = x >> 2; /* x / 4 (unsigned; for signed, rounds toward -inf) */
y = (x << 3) + (x << 1); /* x*10 = x*8 + x*2 */

```

Follow-up they may ask: for signed negative numbers, right-shift is implementation-defined (usually arithmetic shift) and differs from C division, which truncates toward zero: $-7/4 = -1$ but $-7 >> 2 = -2$.

Q8. Check whether an integer is odd, using bitwise ops. Why is $(x \% 2 == 1)$ buggy for negative numbers?

Solution

```

if (x & 1) /* odd */

```

For negative odd x in C99+, $x \% 2$ is -1, not 1, so $x \% 2 == 1$ is false for -3. $x \& 1$ tests the LSB directly and works for all two's-complement values.

Q9. Swap the two nibbles of a byte (e.g. 0xAB becomes 0xBA).

Solution

```

unsigned char swapn(unsigned char b)
{
    return (unsigned char)((b << 4) | (b >> 4));
}

```

For a full 32-bit byte-swap see Q17 (endianness section).

Q10. Compute the parity (1 if the number of set bits is odd) of a 32-bit word using XOR folding.

Solution

```
int parity(unsigned x)
{
    x ^= x >> 16;
    x ^= x >> 8;
    x ^= x >> 4;
    x ^= x >> 2;
    x ^= x >> 1;
    return x & 1;
}
```

Each fold XORs the top half into the bottom half; after 5 folds the LSB is the XOR of all 32 bits. This is literally an XOR reduction tree — the same structure as Verilog's `^x` reduction operator. A very ASIC-flavored question.

Q11. What do the expressions $(x \& -x)$ and $(x \mid (x-1))$ compute? Assume two's complement.

Solution

$x \& -x$ isolates the lowest set bit: for $x = 0b1011000$ the result is $0b0001000$. Because $-x = \sim x + 1$, all bits below the lowest set bit are 0 in both and only that bit is 1 in both. Used for one-hot extraction and priority encoders.

$x \mid (x-1)$ sets all bits below (and including) the lowest set bit: $0b1011000$ becomes $0b1011111$. Useful for building masks.

Q12. Sign-extend a 12-bit two's-complement value (stored in the low 12 bits of an int) to a full 32-bit int. (Directly relevant to datapath design.)

Solution

```
int sext12(int v)                /* v holds a 12-bit value in bits [11:0] */
{
    v &= 0xFFFF;                 /* be safe: keep only 12 bits */
    if (v & 0x800)                /* sign bit (bit 11) set? */
        v |= ~0xFFFF;           /* fill bits [31:12] with 1s */
    return v;
}

/* Branch-free classic: */
int sext12b(int v) { return (v << 20) >> 20; } /* relies on arithmetic >> */
```

The shift trick moves bit 11 into the sign position then arithmetic-shifts back, replicating the sign — exactly what a sign-extension unit does in hardware. Note it relies on implementation-defined arithmetic right shift of signed ints (true on gcc/clang).

Q13. TRICKY — What is the output? (Operator precedence trap)

```
#include <stdio.h>
int main(void)
{
    unsigned x = 4;                /* binary 100 */
    if (x & 1 == 0)
        printf("even\n");
    else
        printf("odd\n");
    return 0;
}
```

Solution

Output: **odd**. Precedence trap: `==` binds tighter than `&`, so this evaluates as `x & (1 == 0) = x & 0 = 0` → false → else branch. Correct code needs parentheses: `(x & 1) == 0`. This exact trap appears constantly in hardware interviews because register-flag tests look exactly like this.

Q14. TRICKY — What is the output on a machine with 32-bit int?

```
#include <stdio.h>
int main(void)
{
    printf("%d\n", 1 << 31 >> 31);          /* signed */
    unsigned u = 1u << 31;
    printf("%u\n", u >> 31);                /* unsigned */
    printf("%d\n", -8 >> 1);                /* signed right shift */
    return 0;
}
```

Solution

Line 1: **-1** (on typical gcc/clang). `1<<31` gives `INT_MIN`'s bit pattern; arithmetic right shift replicates the sign bit → all 1s = -1. (Strictly, `1<<31` on signed int is undefined behaviour — say this!) Line 2: **1** — unsigned right shift is logical, fills zeros. Line 3: **-4** — arithmetic shift of -8 (`...11111000` → `...11111100`). Takeaway: signed vs unsigned shift = arithmetic vs logical shifter in hardware.

Q15. Set the lowest cleared (0) bit of x, and clear the lowest set bit of x.

Solution

```
x | (x + 1);    /* set the lowest 0 bit : 0b1011 -> 0b1111? no: 1011+1=1100, or=1111... */
x & (x - 1);    /* clear the lowest 1 bit: 0b1011 -> 0b1010 */
```

Check `x | (x+1)` with `x = 0b1001`: `x+1 = 0b1010`, `OR = 0b1011` — only the lowest 0 bit changed to 1. And `x&(x-1)` is the Kernighan step from Q3.

Q16. Given two unsigned ints a and b, count how many bits must be flipped to convert a into b.

Solution

```
int flips(unsigned a, unsigned b)
{
    unsigned x = a ^ b;    /* 1s exactly where a and b differ */
    int c = 0;
    while (x) { x &= x - 1; c++; }
    return c;
}
```

XOR marks differing bit positions (this is the Hamming distance); then popcount. In verification, this is how you count bit errors between expected and actual bus values.

Q17. Round an address up to the next multiple of 8 (alignment), assuming 8 is a power of 2. Generalize for any power-of-2 alignment A.

Solution

```
addr = (addr + 7) & ~7u;          /* align up to 8 */
addr = (addr + (A - 1)) & ~(A - 1); /* general, A = 2^k */
```

Adding `A-1` pushes any non-aligned value past the next boundary; masking with `~(A-1)` zeroes the low $\log_2(A)$ bits. Used everywhere: DMA buffer alignment, cache-line alignment, memory-mapped region sizing.

Section 2 — Data Representation, sizeof, Endianness & Padding

Q18. What is the output on a typical 64-bit Linux system (gcc)? Explain each line.

```
#include <stdio.h>
#include <string.h>
int main(void)
```

```

{
    char str[20] = "Arnab";
    int arr[5] = {1,3,5,7,9};
    printf("%zu %zu\n", sizeof(str), strlen(str));
    printf("%zu %zu\n", sizeof(arr), sizeof(arr)/sizeof(arr[0]));
    printf("%zu %zu %zu\n", sizeof(char), sizeof(int), sizeof(void *));
    return 0;
}

```

Solution

Output:

```

20 5
20 5
1 4 8

```

sizeof is a compile-time property of the TYPE (20 bytes allocated, 20 bytes = 5 ints), while strlen counts characters up to '\0' at runtime. Pointers are 8 bytes on a 64-bit machine, 4 on 32-bit — a favorite quick-fire question (this was your Quiz 10 Q4 and practice.c sizeof(void*)).

Q19. Write a C program to determine at runtime whether the machine is little-endian or big-endian. Give both the union method and the char-pointer method. (Guaranteed interview question.)

Solution

```

/* Method 1: char pointer */
int is_little_endian(void)
{
    unsigned int x = 1;
    return *(char *)&x == 1;    /* LSB stored at lowest address? */
}

/* Method 2: union */
int is_little_endian_u(void)
{
    union { unsigned int i; unsigned char c[4]; } u = { .i = 1 };
    return u.c[0] == 1;
}

```

Little-endian (x86, most ARM): least-significant byte at the LOWEST address, so byte 0 of the int 0x00000001 is 0x01. Big-endian (network order, some DSPs): MSB first, byte 0 would be 0x00. Follow-up: for 0x12345678 at address 0x100, little-endian memory is 78 56 34 12; big-endian is 12 34 56 78.

Q20. Write functions to byte-swap a 16-bit and a 32-bit value (i.e., convert between endiannesses, like htons/htonl).

Solution

```

unsigned short swap16(unsigned short x)
{
    return (unsigned short)((x << 8) | (x >> 8));
}

unsigned int swap32(unsigned int x)
{
    return ((x & 0x000000FFu) << 24) |
           ((x & 0x0000FF00u) << 8) |
           ((x & 0x00FF0000u) >> 8) |
           ((x & 0xFF000000u) >> 24);
}

```

Needed whenever your little-endian CPU talks to a big-endian bus/protocol (network order, some sensor payloads). ARM even has a single instruction (REV) for this — the C above is what compilers turn into it.

Q21. Predict sizeof for each struct on a typical 64-bit system, explain the padding, and reorder s1 to minimize its size.

```

struct s1 { char a; int b; char c; };
struct s2 { char a; char c; int b; };
struct s3 { double y; int x[2]; };      /* from your Quiz 12! */
struct s4 { char a; double d; char b; };

```

Solution

Rules: each member is aligned to its own size; total struct size is padded to a multiple of the largest member alignment.

```

s1: a(1) +pad(3) b(4) c(1) +pad(3)      = 12
s2: a(1) c(1) +pad(2) b(4)              = 8   <- reordered version of s1
s3: y(8) x(8)                          = 16   (no padding needed)
s4: a(1)+pad(7) d(8) b(1)+pad(7)        = 24

```

Reordering members from largest to smallest minimizes padding (s1→s2 saves 4 bytes). In register-map structs you instead FORBID padding with `__attribute__((packed)) / #pragma pack(1)`, at the cost of slower unaligned access. Interviewers often ask: 'why does the compiler pad at all?' — because aligned loads/stores are single bus transactions; unaligned ones may need two.

Q22. Define a struct with bit-fields to model an 8-bit status register with fields: READY(1 bit), ERROR(1 bit), MODE(2 bits), COUNT(4 bits). Then state why production register maps usually avoid bit-fields.

Solution

```

typedef struct {
    unsigned char READY : 1;
    unsigned char ERROR : 1;
    unsigned char MODE  : 2;
    unsigned char COUNT : 4;
} status_t;

status_t s;
s.MODE = 3;      /* compiler does the mask+shift for you */

```

Avoided in real register maps because the C standard leaves bit ORDER within the storage unit and padding implementation-defined — the same struct can map READY to bit 0 on one compiler and bit 7 on another. MISRA-style codebases therefore use explicit masks/shifts (Q6) for portability.

Q23. TRICKY — What does this print, and why? (Signed/unsigned comparison trap)

```

#include <stdio.h>
int main(void)
{
    int a = -1;
    unsigned int b = 1;
    if (a > b) printf("-1 > 1  (?)\n");
    else     printf("-1 <= 1\n");
    printf("%u\n", (unsigned)a);
    return 0;
}

```

Solution

Output: **-1 > 1 (?)** then **4294967295**. In the comparison, the usual arithmetic conversions convert the signed -1 to unsigned, giving `0xFFFFFFFF = 4294967295`, which is `> 1`. This bites constantly in hardware code where register reads are unsigned and loop counters are signed. Fix: never mix; or cast explicitly.

Q24. TRICKY — What is the output? (char range / truncation, ties to your Quiz 3)

```

#include <stdio.h>
int main(void)
{
    char c = 255;          /* char is signed on x86 gcc */
    unsigned char u = 255;
    c = c + 1; u = u + 1;
    printf("%d %d\n", c, u);
}

```

```

char f(void);           /* returns 'A' + 256 truncated? */
int big = 0x141;        /* 321 */
char t = (char)big;
printf("%d\n", t);
return 0;
}

```

Solution

First line: **0 0**. Signed char 255 is actually -1 (implementation-defined but typical); $-1+1 = 0$. Unsigned char wraps $255+1 \rightarrow 0$ by modulo-256 arithmetic (well defined). Second: $0x141$ truncated to 8 bits keeps only the low byte $0x41 = 65$ ('A'). Truncation = keeping the low N bits — identical to assigning a wide wire to a narrow one in Verilog, which is why ASIC interviewers ask it.

Q25. Using a union, show how to inspect the 4 bytes of the float 1.0f (type punning). What would you expect on a little-endian machine given IEEE-754 single precision?

Solution

```

#include <stdio.h>
int main(void)
{
    union { float f; unsigned char b[4]; unsigned int u; } pun;
    pun.f = 1.0f;
    printf("0x%08X\n", pun.u);           /* 0x3F800000 */
    for (int i = 0; i < 4; i++) printf("%02X ", pun.b[i]);
    /* little-endian prints: 00 00 80 3F */
    return 0;
}

```

IEEE-754: $1.0f = \text{sign } 0, \text{exponent } 127 (0111_1111), \text{mantissa } 0 \rightarrow 0x3F800000$. On little-endian, $b[0]$ is the LSB $0x00$ and $b[3]$ is $0x3F$. Unions are the standard C way to reinterpret bits (type punning); casting pointers instead violates strict aliasing. This one question tests floats, unions AND endianness together — high-yield.

Section 3 — Pointers, volatile & Memory-Mapped Registers

Q26. What does the volatile keyword do, and give the three classic situations in hardware programming where it is mandatory. What bug appears if you forget it?

Solution

volatile tells the compiler the value can change outside normal program flow, so it must NOT cache it in a register, reorder, or delete 'redundant' reads/writes — every access in the source becomes a real load/store on the bus.

Mandatory for: (1) memory-mapped hardware/peripheral registers, (2) variables shared with an interrupt service routine (ISR), (3) variables shared between threads/cores without other synchronization.

```

volatile int flag = 0;
while (flag == 0) { } /* wait for ISR to set flag */
/* WITHOUT volatile: compiler reads flag once, loops forever
   on the cached value -> classic 'works with -O0, hangs with -O2' bug */

```

Q27. Write C code to (a) write 0x5A to an 8-bit register at address 0x40021000 and (b) poll bit 3 of a 32-bit status register at 0x40021004 until it becomes 1.

Solution

```

#define CTRL (*(volatile unsigned char *)0x40021000u)
#define STATUS (*(volatile unsigned int *)0x40021004u)

CTRL = 0x5A; /* (a) single 8-bit store to the bus */

while ((STATUS & (1u << 3)) == 0) /* (b) re-reads register every loop */
    ; /* because of volatile */

```

The pattern: cast the literal address to a pointer-to-volatile of the register's width, dereference it. The macro makes it read like a variable. This is THE bread-and-butter question for any role touching hardware.

Q28. Explain the difference between these four declarations. (Asked verbatim in many interviews.)

```
const int *p;           /* 1 */
int const *p;           /* 2 */
int * const p;          /* 3 */
const volatile int *p;  /* 4 */
```

Solution

(1) and (2) are identical: pointer to const int — the pointed-to VALUE can't be modified through p, but p itself can be repointed. (3) const pointer to int — the value can change, but p is fixed after initialization. (4) pointer to a const volatile int: your code may not write it, but hardware may change it at any time — the exact type for a read-only status register: `const volatile uint32_t *STATUS = (uint32_t*)0x80102040;`. Rule of thumb: read right-to-left from the variable name.

Q29. TRICKY — Which of these functions is broken, and what is the bug called? (Your Quiz 13 Q4, a real interview staple)

```
int *f1(void) { int x = 10; return &x; }
int *f2(void) { int *p;      *p = 10; return p; }
int *f3(void) { int *p = malloc(sizeof(int)); *p = 10; return p; }
```

Solution

f1: returns the address of a LOCAL — the stack frame dies at return, leaving a **dangling pointer**; dereferencing it is undefined behaviour. f2: writes through an UNINITIALIZED (wild) pointer — undefined behaviour immediately. f3: correct — heap memory outlives the function (caller must free it, else a memory leak). Only f3 is safe.

Q30. Trace the output. (Pointer-to-pointer, in the style of your Quiz 13 Q3)

```
#include <stdio.h>
void f(int x, int *py, int **ppz)
{
    **ppz += 2;    /* changes the int that ppz ultimately points to */
    *py  += 1;     /* changes the int py points to */
    x    += 5;     /* local copy only */
}
int main(void)
{
    int x = 10;
    int *py = &x;
    int **ppz = &py;
    f(x, py, ppz);
    printf("%d\n", x);
    return 0;
}
```

Solution

Both `**ppz` and `*py` reach the same variable `x` in `main`: $x = 10 + 2 + 1 = 13$. The `x += 5` in `f` only changes the pass-by-value copy. Method: draw the pointer diagram (`ppz → py → x`) before tracing — never trace multi-level pointers in your head.

Q31. Implement `strlen` without using the `[]` operator (from your practice sheet), and also implement `memcpy`. State the classic `memcpy` interview follow-up.

Solution

```
size_t my_strlen(const char *s)
{
    const char *p = s;
    while (*p) p++;
}
```

```

    return (size_t)(p - s);      /* pointer difference = element count */
}

void *my_memcpy(void *dst, const void *src, size_t n)
{
    unsigned char *d = dst;
    const unsigned char *s = src;
    while (n--) *d++ = *s++;
    return dst;
}

```

Follow-up: 'what if the regions overlap?' — memcpy has undefined behaviour for overlap; you must use memmove, which copies backwards when dst > src. Being ready with that answer is expected.

Q32. TRICKY — What is the output? (Pointer arithmetic scales by type size — from your practice.c)

```

#include <stdio.h>
int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;
    char *c = (char *)a;

    printf("%d\n", *(p + 2));
    printf("%d\n", *(int *)(c + 8));
    printf("%td\n", (char*)(p+1) - (char*)p);
    char str[] = "CP is the best course";
    char *q = str; q += 6;
    printf("%s\n", q);
    return 0;
}

```

Solution

```

30
30
4
the best course

```

p+2 advances 2 INTS (8 bytes) → a[2]=30. c+8 advances 8 CHARS (8 bytes) → same location, reinterpreted as int → 30. (p+1)-(p) in char terms = sizeof(int) = 4. Pointer arithmetic always scales by the pointed-to type's size — this is why an int* and char* to the same address behave differently, and why byte-addressing vs word-addressing matters when you map C onto a bus.

Q33. An interrupt sets a shared counter. Explain why this main-loop code can still lose updates even WITH volatile, and name the concept.

```

volatile unsigned count;
void ISR(void) { count++; }
int main(void) { ...; count++; ... } /* also increments */

```

Solution

count++ compiles to read-modify-write (load, add, store). If the ISR fires between main's load and store, its increment is overwritten — a **race condition**. volatile forces real memory accesses but does NOT make them atomic. Fix on a microcontroller: disable interrupts around the update (critical section), or use hardware atomic instructions. Distinguishing volatile from atomicity is a favorite senior-level probe.

Section 4 — Output-Tracing Traps (quiz style)

These follow the exact style of your course quizzes — trace on paper, no compiler. They test the traps interviewers use to see if you actually know C or just recognize it.

Q34. What is the output? (Pre/post increment — Quiz 2 style)

```
#include <stdio.h>
int main(void)
{
    int x = 5;
    int y = x;
    printf("x = %d, y = %d\n", x, y++);
    printf("y = %d\n", y);
    int i = 5;
    int j = ++i + i++;      /* what about this? */
    printf("maybe %d\n", j);
    return 0;
}
```

Solution

Line 1: **x = 5, y = 5** — postfix `y++` yields the OLD value, increments after. Line 2: **y = 6**. Line 3: **unspecified/undefined** — modifying `i` twice without a sequence point is undefined behaviour; different compilers print different numbers. Saying 'this is UB, I would never write it' is the correct interview answer, not computing 12 or 13.

Q35. What is the output? (Empty statement trap — Quiz 4/7 style)

```
#include <stdio.h>
int main(void)
{
    int x = 11;
    if (x == 10);
        printf("A");
    if (x = 10)
        printf("B");
    printf(" x=%d\n", x);
    return 0;
}
```

Solution

Output: **AB x=10**. The semicolon after the first `if` is an empty body, so `printf("A")` runs unconditionally. The second `if` uses ASSIGNMENT (`=`), which stores 10 and evaluates to 10 (truthy) → B prints and `x` is silently corrupted. Both are top-5 real-world C bugs; many hardware teams mandate 'Yoda conditions' (`10 == x`) or compiler warnings to catch them.

Q36. What is the output? (Recursion print order — Quiz 7)

```
#include <stdio.h>
void test(int n)
{
    if (n > 0) {
        test(n - 1);
        printf("%d ", n);
    }
}
int main(void) { test(4); return 0; }
```

Solution

Output: **1 2 3 4**. The print happens AFTER the recursive call, so the deepest call (`n=1`) prints first as the stack unwinds. If `printf` came before the recursion, output would be 4 3 2 1. Understanding call-stack order matters for hardware people too — it is the same LIFO behavior as a return-address stack.

Q37. What is the output? (Recursive digit-sum — Quiz 6, also useful for checksums)

```
#include <stdio.h>
int f(int x)
{
    if (x == 0) return 0;
    return (x % 10) + f(x / 10);
}
int main(void) { printf("%d\n", f(65982)); return 0; }
```

Solution

Digit sum: $6+5+9+8+2 = 30$. Trace: $f(65982)=2+f(6598)=2+8+f(659)=\dots$ The `%10 / /10` pair peels the last digit — the decimal analog of masking the low nibble and shifting.

Q38. What is the output? (Array decays to pointer inside functions — Quiz 9)

```
#include <stdio.h>
void size(int arr[])
{
    printf("in func: %zu\n", sizeof(arr));    /* 64-bit machine */
}
int main(void)
{
    int arr[5] = {1,3,5,7,9};
    printf("in main: %zu\n", sizeof(arr));
    size(arr);
    return 0;
}
```

Solution

Output: **in main: 20, in func: 8**. An array parameter is really a pointer (`int*`), so `sizeof` gives pointer size (8 on 64-bit). Consequence: a function can never recover an array's length from the pointer alone — you must pass the length separately. This is why every driver API takes (`buf, len`).

Q39. What is the output? (String library behavior — Quiz 10)

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    char str [30] = "Arnab Paul";
    char str2[30] = "123";
    strcat(str2, str);
    strcpy(str2, str);
    printf("%s\n", str2);
    printf("%d\n", strcmp("ABC", "AB12"));    /* sign only */
    return 0;
}
```

Solution

Line 1: **Arnab Paul** — `strcat` makes `str2 = "123Arnab Paul"`, but `strcpy` then **OVERWRITES** it entirely with "Arnab Paul". Line 2: **positive** (e.g. 1) — comparison stops at the first differing char: 'C'(67) vs '1'(49), and $67-49 > 0$. `strcmp` returns the sign of the first difference, not a full lexicographic 'distance'.

Q40. What is the output? (enum arithmetic — Quiz 16)

```
#include <stdio.h>
enum month { jan, feb, mar, apr = 5, may, jun };
int main(void)
{
    printf("%d\n", jan + mar + may);
    return 0;
}
```

Solution

`jan=0, feb=1, mar=2`, then `apr` is forced to 5, so `may=6, jun=7`. Sum = $0+2+6 = 8$. Enums auto-increment from the last explicit value — the standard way to encode FSM states and register field values in C models of hardware.

Q41. What is the output? (2-D array addressing — Quiz 16 Q6 style)

```
#include <stdio.h>
int main(void)
{
    char b[3][4] = {"ABC", "DEF", "GHI"};
```

```

/* assume b starts at address 0x74 */
printf("%p ", (void*)(*(b+2)+2)); /* address */
printf("%c\n", *(*(b+2)+2));      /* value */
return 0;
}

```

Solution

b+2 skips two ROWS of 4 bytes = 8 bytes → 0x7C. *(b+2) decays that row to a char* at 0x7C; +2 → **0x7E**. Dereferencing gives row "GHI" index 2 → 'I'. Formula: address of b[i][j] = base + (i*NCOLS + j)*sizeof(elem) — the same row-major flattening you use for memory arrays in RTL.

Q42. What is the output? (Macro pitfall — asked in nearly every embedded interview)

```

#include <stdio.h>
#define SQR(x) x * x
#define SQR2(x) ((x) * (x))
int main(void)
{
    printf("%d\n", SQR(3 + 1));
    printf("%d\n", 16 / SQR2(2));
    int i = 3;
    printf("%d\n", SQR2(i++)); /* danger! */
    return 0;
}

```

Solution

Line 1: SQR(3+1) expands to 3 + 1 * 3 + 1 = **7**, not 16 — always parenthesize macro args AND the whole body. Line 2: **4** — SQR2 is safe here. Line 3: expands to ((i++) * (i++)) — i modified twice, **undefined behaviour**; macros evaluate arguments multiple times, so never pass expressions with side effects. This is also the argument for inline functions over macros.

Q43. What is the output, and what is the difference between the two loops? (Loop-boundary reasoning — Quiz 8)

```

#include <stdio.h>
int main(void)
{
    int i = 0, sum = 0;
    for (i = 0; i < 10; i++) sum += i;
    printf("%d %d\n", sum, i);

    unsigned char n = 0;
    int guard = 0;
    while (n < 256 && guard < 1000) { n++; guard++; } /* trap */
    printf("%d %d\n", n, guard);
    return 0;
}

```

Solution

Line 1: sum = 0+...+9 = **45**, and i exits the loop as **10**. Line 2: an unsigned char can never reach 256 — it wraps 255→0 — so n < 256 is ALWAYS true and only the guard stops it: prints **232 1000** (1000 mod 256 = 232). Counter-width vs comparison-range mismatches are exactly the off-by-one/wrap bugs that kill hardware counters too.

Section 5 — Hardware-Flavored Coding Problems

Q44. Write a C model of a 4-bit up-counter with synchronous clear and enable. One function call = one clock edge. (Bridges C and RTL thinking.)

Solution

```

typedef struct { unsigned char q; } counter4_t;

void counter_clk(counter4_t *c, int clr, int en)

```

```

{
    if (clr)        c->q = 0;
    else if (en)    c->q = (c->q + 1) & 0xF;    /* wrap at 16 like 4-bit HW */
}

```

Key points: state lives in a struct (like flip-flops), the mask & 0xF models the 4-bit width (C ints don't wrap at 4 bits by themselves), and priority `clr > en` mirrors the if/else priority in an always block.

Q45. Implement a parameterizable one-hot checker: return 1 if exactly one bit of x is set.

Solution

```

int is_onehot(unsigned x)
{
    return x != 0 && (x & (x - 1)) == 0;
}

```

Identical to the power-of-2 test (Q2) — 'one-hot' is just the hardware name. FSM state vectors and grant vectors from arbiters are checked this way in verification code.

Q46. Compute log2 of a power-of-2 unsigned int (i.e., find the position of the single set bit).

Solution

```

int log2_pow2(unsigned x)          /* precondition: x is a power of 2 */
{
    int n = 0;
    while (x >= 1) n++;
    return n;
}
/* Faster: binary search on masks */
int log2f(unsigned x)
{
    int n = 0;
    if (x & 0xFFFF0000u) { n += 16; x >= 16; }
    if (x & 0x0000FF00u) { n += 8;  x >= 8;  }
    if (x & 0x000000F0u) { n += 4;  x >= 4;  }
    if (x & 0x0000000Cu) { n += 2;  x >= 2;  }
    if (x & 0x00000002u) { n += 1;  }
    return n;
}

```

The binary-search version is a software priority encoder — describing it that way in an ASIC interview shows you connect the two worlds. Used for converting one-hot grants to binary indices.

Q47. Implement a software 'shift register': push a new bit into the LSB of an 8-bit value, and detect the pattern 0b1011 in the stream. (Serial protocols / sequence detectors.)

Solution

```

unsigned char sr = 0;

int push_and_check(int bit)
{
    sr = (unsigned char)((sr << 1) | (bit & 1));
    return (sr & 0x0F) == 0x0B;    /* last 4 bits == 1011 ? */
}

```

This is the C equivalent of a Mealy sequence detector: shift in the sampled bit, compare a masked window. UART framing, sync-word hunting and CRC bit-serial logic all look like this.

Q48. Given a 32-bit word read from a little-endian memory containing four packed 8-bit sensor samples (oldest in byte 0), extract sample k (0..3) and sign-extend it to int.

Solution

```

int get_sample(unsigned word, int k)
{

```

```

    unsigned char b = (word >> (8 * k)) & 0xFF;    /* extract byte k */
    return (int)(signed char)b;                    /* sign-extend 8->32 */
}

```

Two idioms combined: byte-lane extraction by shifting multiples of 8 (a byte mux), and sign-extension via cast to signed char then widening. On big-endian storage the shift would be $8*(3-k)$ — always state your endianness assumption.

Q49. Write a function that reverses the BYTES of a buffer in place (e.g., for endianness fixing of a multi-word payload).

Solution

```

void reverse_bytes(unsigned char *buf, int n)
{
    int i = 0, j = n - 1;
    while (i < j) {
        unsigned char t = buf[i];
        buf[i] = buf[j];
        buf[j] = t;
        i++; j--;
    }
}

```

Classic two-pointer in-place reversal — also the building block for the 'reverse words in a string' interview question (reverse the whole buffer, then each word).

Q50. Implement a simple parity-based error detector: given a 7-bit data value, return the 8-bit codeword with even parity in the MSB; and a checker that validates a received byte.

Solution

```

unsigned char encode(unsigned char d)    /* d in [0,127] */
{
    unsigned p = d;
    p ^= p >> 4; p ^= p >> 2; p ^= p >> 1; /* XOR-fold to get parity */
    return (unsigned char)(((p & 1) << 7) | d); /* even parity bit in MSB */
}

int check(unsigned char cw)             /* 1 = OK */
{
    unsigned p = cw;
    p ^= p >> 4; p ^= p >> 2; p ^= p >> 1;
    return (p & 1) == 0;                /* total parity must be even */
}

```

Even parity: total number of 1s including the parity bit is even, so the whole received codeword must XOR-reduce to 0. This tiny example generalizes to Hamming/ECC questions that follow it in interviews.

Section 6 — Structs, Unions & Register Maps

Q51. Model a peripheral's register block (CTRL at offset 0x0, STATUS at 0x4, DATA at 0x8, reserved gap, IRQ_EN at 0x10) as a C struct at base address 0x40020000, and use it to enable bit 0 of IRQ_EN.

Solution

```

typedef struct {
    volatile unsigned int CTRL;    /* 0x00 */
    volatile unsigned int STATUS; /* 0x04 */
    volatile unsigned int DATA;  /* 0x08 */
    unsigned int RESERVED0;       /* 0x0C - keep offsets correct! */
    volatile unsigned int IRQ_EN; /* 0x10 */
} periph_t;

#define PERIPH ((periph_t *)0x40020000u)

```

```
PERIPH->IRQ_EN |= 1u; /* read-modify-write bit 0 */
```

Three graded details: every real register member is volatile; RESERVED placeholders preserve offsets across gaps (off-by-one-register bugs are silent and deadly); the struct must have no padding — all members here are 4-byte aligned uint32s so it's naturally packed. This is how every vendor SDK (STM32, etc.) defines GPIO_TypeDef.

Q52. What is the difference between a struct and a union? Predict the sizeof each, and give the standard hardware use of a union.

```
struct S { unsigned int w; unsigned char b[4]; };
union U { unsigned int w; unsigned char b[4]; };
```

Solution

A struct lays members out sequentially (sizeof S = 8); a union OVERLAYS all members on the same storage — its size is the largest member (sizeof U = 4). Hardware uses: (1) access the same register as one 32-bit word OR as individual bytes/fields, (2) endianness inspection (Q16), (3) type punning float↔bits (Q23). Writing u.w then reading u.b[] is exactly reading a register's byte lanes.

Q53. TRICKY — What is the output? (struct copy vs pointer, Quiz 15 spirit)

```
#include <stdio.h>
typedef struct { int id; int regs[2]; } dev_t;

void update_v(dev_t d) { d.regs[0] = 99; } /* by value */
void update_p(dev_t *d) { d->regs[0] = 99; } /* by pointer */

int main(void)
{
    dev_t a = {1, {10, 20}};
    dev_t b = a; /* struct assignment COPIES everything */
    b.regs[0] = 55;
    update_v(a);
    printf("%d %d\n", a.regs[0], b.regs[0]);
    update_p(&a);
    printf("%d\n", a.regs[0]);
    return 0;
}
```

Solution

Output: **10 55** then **99**. Struct assignment b = a deep-copies the embedded array (unlike raw arrays, which can't be assigned). update_v receives a copy, so a is untouched; update_p mutates through the pointer. Passing large structs by pointer (plus const if read-only) is also the performance-correct habit.

Q54. Compute the offset of member DATA within periph_t (Q39) without offsetof, then show the standard offsetof macro trick and why it works.

Solution

```
size_t off = (size_t)&(((periph_t *)0)->DATA); /* = 8 */
/* The library macro is essentially: */
#define my_offsetof(T, m) ((size_t)&(((T *)0)->m))
```

Casting 0 to T* and taking the ADDRESS of a member computes base(0) + offset without any dereference, so the null pointer is never accessed. Interviewers use this to test whether you understand that &p->m is pure address arithmetic. (Strictly it's UB by the letter of the standard, which is why the real offsetof is compiler-provided.)

Q55. Explain typedef vs #define for types, with the classic pointer trap.

```
#define PTR1 int *
typedef int * PTR2;

PTR1 a, b;
PTR2 c, d;
```

Solution

The #define is textual: `PTR1 a, b;` expands to `int *a, b;` — `a` is `int*` but **`b` is plain `int`!** The typedef creates a real type, so both `c` and `d` are `int*`. Rule: always typedef types; reserve #define for constants and code-generation.

Section 7 — Rapid-Fire Conceptual Questions (verbal round)

Q56. What are the three distinct uses of the static keyword in C?

Solution

(1) static local variable: retains its value between calls, initialized once, lives in `.data/.bss` not the stack — how you hold persistent state (like a flip-flop) inside a function. (2) static global variable: visible only within its `.c` file (internal linkage) — information hiding. (3) static function: same file-local linkage for functions — keeps driver internals out of the global namespace.

Q57. Stack vs heap: where do these live, and what typically goes wrong with each in embedded systems?

```
int g;                /* ? */
static int s = 5;     /* ? */
void f(void){
    int l;            /* ? */
    static int c;      /* ? */
    int *p = malloc(40); /* p? *p? */
}
```

Solution

`g` and `s`: `.bss/.data` (globals). `l`: stack (dies at return). `c`: `.bss` (static storage, persists). `p` itself: stack; the 40 bytes it points to: heap. Embedded failure modes: stack overflow from deep recursion/large locals (often no MMU to catch it), and heap fragmentation/leaks — which is why many firmware standards ban `malloc` after init.

Q58. Difference between a macro function and an inline function? When must you use which?

Solution

Macros are preprocessor text substitution: no type checking, arguments possibly evaluated multiple times (`SQR(i++)` bug, Q33), invisible in the debugger. inline functions are real functions with type checking and single evaluation that the compiler may expand in place — prefer them for logic. Macros remain necessary for: compile-time constants used in array sizes/switch labels, token pasting, and things functions cannot do (e.g., `#define REG(addr) (*(volatile uint32_t*)(addr))`).

Q59. What happens before `main()` runs, and what is the difference between the compiler, assembler, linker and loader?

Solution

Compiler translates C to assembly (per translation unit); assembler produces object files; linker resolves symbols across objects/libraries and assigns final addresses per the linker script (`.text` to flash, `.data/.bss` to RAM); loader (or on-chip boot) places the image in memory. Startup code before `main` copies `.data` initial values from flash to RAM, zeroes `.bss`, sets up the stack pointer and vector table — a firmware-specific answer interviewers appreciate.

Q60. Why must an ISR (interrupt service routine) be short, and why can't it take parameters or return a value?

Solution

An ISR is invoked by hardware, not by a caller — there is nobody to pass parameters or receive a return value, so its signature is `void isr(void)`. It must be short because it runs with interrupts (of same/lower priority) blocked: long ISRs cause missed events and jitter. Best practice: set a volatile flag / push into a queue, do the work in the main loop. Shared data needs volatile plus atomicity protection (Q28).

Q61. What is the output of `sizeof('A')` in C, and how does it differ in C++? What about `sizeof(1.5)` and `sizeof(1.5f)`?

Solution

In C, a character constant has type `int`, so `sizeof('A') = 4`. (In C++ it's `char` → 1 — a famous C/C++ divergence.) `sizeof(1.5) = 8` (double — floating literals default to double), `sizeof(1.5f) = 4` (float). Corollary: `float f = 1.5;` silently computes in double and converts, which matters on FPUs without double support.

Q62. Explain what 'undefined behaviour' means and list five UB examples that appeared in this set.

Solution

UB = the standard imposes NO requirements; the compiler may assume it never happens and optimize accordingly — the code may 'work', crash, or break only at -O2. Examples from this set: (1) shifting a signed int into/past the sign bit ($1 < 31$), (2) shifting by $\geq$ the type width, (3) modifying a variable twice without a sequence point (`i++ * i++`), (4) dereferencing a dangling or wild pointer (Q27), (5) signed integer overflow (`INT_MAX+1`). Being able to SPOT UB is a strong hire signal.

Q63. A colleague's code works at -O0 but hangs at -O2. The code polls a status flag written by an ISR. Diagnose it in one sentence, and name one more bug class with the same symptom.

Solution

The flag isn't declared volatile, so the optimizer hoisted the load out of the loop and spins on a stale register copy — add volatile. Same symptom also arises from UB in the code (the optimizer exploited it) or from missing memory barriers on multi-core systems. 'Works only without optimization' should always trigger the volatile/UB checklist.

Q64. What is the difference between `++*p`, `*p++`, and `(*p)++` ?

Solution

`++*p`: increments the VALUE pointed to (pre-increment), `p` unchanged. `*p++`: postfix `++` binds to `p` — yields `*p`, then advances the POINTER to the next element (the idiom inside `memcpy`: `*d++ = *s++`). `(*p)++`: yields the value, then increments the pointed-to VALUE. Precedence: postfix `++` > unary `*` — when in doubt, parenthesize.

Q65. Final boss — predict the complete output on 64-bit little-endian gcc, combining everything.

```
#include <stdio.h>
int main(void)
{
    unsigned int r = 0xDEADBEEF;
    unsigned char *p = (unsigned char *)&r;

    printf("%02X\n", p[0]);           /* endianness      */
    printf("%X\n", (r >> 8) & 0xFF); /* byte extraction */
    printf("%d\n",  !(r & (1u << 31))); /* MSB test       */
    r = (r & ~0xFF000000u) | (0x5Au << 24); /* field insert    */
    printf("%08X\n", r);
    printf("%d\n", (int)sizeof(r + 0.0)); /* promotion      */
    return 0;
}
```

Solution

```
EF
BE
1
5AADBEEF
8
```

`p[0]` is the lowest-addressed byte = LSB on little-endian = EF. `(r>>8)&0xFF` picks byte 1 = BE. Bit 31 of 0xD... (1101...) is set → !! normalizes to 1. The insert clears the top byte then ORs in 0x5A → 5AADBEEF. `r + 0.0` promotes to double → `sizeof = 8`. If you got all five, you're in good shape for the interview.

Section 8 — More Bit Tricks & Digital-Design Idioms

Q66. Convert a binary number to Gray code, and Gray code back to binary. Why does Gray code matter in digital design?

Solution

```
unsigned bin2gray(unsigned b)
{
    return b ^ (b >> 1);
}

unsigned gray2bin(unsigned g)
{
    unsigned b = 0;
    while (g) { b ^= g; g >>= 1; } /* XOR of all suffixes */
    return b;
}

/* log-time version of gray2bin: */
unsigned gray2bin_fast(unsigned g)
{
    g ^= g >> 16; g ^= g >> 8; g ^= g >> 4; g ^= g >> 2; g ^= g >> 1;
    return g;
}
```

Gray code changes exactly ONE bit between consecutive values, so a value sampled mid-transition is off by at most 1 — which is why asynchronous FIFO read/write pointers crossing clock domains (CDC) are always Gray-coded. `bin2gray` is one XOR; `gray2bin` is a prefix-XOR (each binary bit = XOR of all Gray bits above it). Expect this question in ANY interview that mentions FIFOs or CDC.

Q67. Implement 32-bit rotate-left and rotate-right by *n* positions (a barrel shifter in C).

Solution

```
unsigned rotl(unsigned x, int n)
{
    n &= 31; /* keep shift in 0..31 */
    return (x << n) | (x >> ((32 - n) & 31));
}

unsigned rotr(unsigned x, int n)
{
    n &= 31;
    return (x >> n) | (x << ((32 - n) & 31));
}
```

Rotation = the bits that fall off one end re-enter the other; C has no rotate operator so you OR the two shifted halves. The `& 31` guards matter: shifting a 32-bit value by 32 is undefined behaviour (when `n = 0`, `32-n = 32`!). Compilers recognize this exact pattern and emit a single ROR/ROL instruction.

Q68. Swap all even-positioned and odd-positioned bits of a 32-bit value (bit 0 with bit 1, bit 2 with bit 3, ...).

Solution

```
unsigned swap_even_odd(unsigned x)
{
    return ((x & 0xAAAAAAAAu) >> 1) | ((x & 0x55555555u) << 1);
}
```

Mask the odd bits (0xAAAAAAAA = 1010...) and shift them right; mask the even bits (0x55555555 = 0101...) and shift them left; OR together. This is the first stage of the divide-and-conquer bit reversal from Q5 — interviewers often ask it standalone.

Q69. Round an arbitrary 32-bit unsigned number UP to the next power of 2 (e.g., 17 becomes 32; 32 stays 32).

Solution

```
unsigned next_pow2(unsigned x)
{
    x--;
    x |= x >> 1;
    x |= x >> 2;
    x |= x >> 4;
    x |= x >> 8;
    x |= x >> 16;
    return x + 1;
}
```

The OR-cascade smears the most significant set bit downward until the number is 0b0111...1, then +1 carries into a single bit. Used to size FIFO depths, hash tables, and address decoders, all of which want power-of-2 sizes. (Caveat: x = 0 returns 0; handle separately if needed.)

Q70. Count leading zeros (CLZ) and trailing zeros (CTZ) of a nonzero 32-bit value with plain C.

Solution

```
int clz(unsigned x) /* leading zeros */
{
    int n = 0;
    if (!(x & 0xFFFF0000u)) { n += 16; x <<= 16; }
    if (!(x & 0xFF000000u)) { n += 8; x <<= 8; }
    if (!(x & 0xF0000000u)) { n += 4; x <<= 4; }
    if (!(x & 0xC0000000u)) { n += 2; x <<= 2; }
    if (!(x & 0x80000000u)) { n += 1; }
    return n;
}

int ctz(unsigned x) /* trailing zeros */
{
    return 31 - clz(x & (unsigned)-(int)x); /* isolate lowest bit, find it */
}
```

CLZ is a leading-zero counter / normalization unit (used in floating-point and priority encoding); CTZ finds the index of the lowest set bit — i.e., which interrupt fired in a pending register. Real CPUs have CLZ instructions; this binary-search form is what you write when they don't.

Q71. Compute min and max of two ints WITHOUT any branch (if/else or ternary). Also give abs(x) without branching.

Solution

```
int min(int a, int b) { return b ^ ((a ^ b) & -(a < b)); }
int max(int a, int b) { return a ^ ((a ^ b) & -(a < b)); }

int myabs(int x)
{
    int m = x >> 31; /* 0 if x>=0, all-1s (-1) if x<0 */
    return (x + m) ^ m; /* = (x ^ m) - m as well */
}
```

```
}
```

$-(a < b)$ is 0 or 0xFFFFFFFF, acting as a hardware-style select mask: $\text{mask}=0$ gives $b^0 \dots = b$ wait — trace min: if $a < b$, $\text{mask}=-1$, $\text{result} = b \wedge (a \wedge b) = a$. If not, $\text{mask}=0$, $\text{result} = b$. For abs: for negative x , $m=-1$, so $(x-1)^{-1} = -x$ (two's-complement identity $\sim(x-1) = -x$). Branchless code avoids pipeline flushes and mirrors mux-based datapaths — say that in the interview.

Q72. Detect (a) whether two ints have opposite signs, and (b) whether the addition a+b of two ints overflowed, without invoking UB.

Solution

```
int opposite_signs(int a, int b)
{
    return (a ^ b) < 0;          /* sign bits differ -> XOR is negative */
}

int add_overflows(int a, int b)    /* check BEFORE adding */
{
    if (b > 0 && a > INT_MAX - b) return 1; /* would overflow up */
    if (b < 0 && a < INT_MIN - b) return 1; /* would overflow down */
    return 0;
}

/* Unsigned wrap detection AFTER adding (well-defined): */
unsigned s = ua + ub;
int wrapped = s < ua;          /* carry-out happened */
```

Signed overflow itself is UB, so you must test the operands beforehand. Unsigned arithmetic wraps mod 2^{32} by definition, and the result being smaller than an operand is exactly the carry-out flag — the same signal your adder produces in RTL.

Q73. Multiply two 8-bit unsigned numbers using only shifts and adds (model of a shift-add multiplier).

Solution

```
unsigned mul8(unsigned char a, unsigned char b)
{
    unsigned result = 0;
    while (b) {
        if (b & 1)
            result += a; /* add shifted multiplicand if bit set */
        a <<= 1; /* next partial product */
        b >>= 1;
    }
    return result;
}
```

Each iteration handles one bit of the multiplier: examine LSB, conditionally accumulate, shift. This is literally the sequential shift-add multiplier from digital design class — writing it in C shows you understand the algorithm you'd implement in RTL, which is exactly the point of the question.

Q74. Reverse the DECIMAL digits of an integer (e.g., 1234 becomes 4321), and check whether a number is a palindrome. Watch the overflow.

Solution

```
int reverse_digits(int x)          /* assume x >= 0 */
{
    long r = 0;                    /* wider type to catch overflow */
    while (x) {
        r = r * 10 + x % 10;
        x /= 10;
    }
    return (r > INT_MAX) ? -1 : (int)r; /* signal overflow */
}
```

```
int is_palin(int x) { return x >= 0 && x == reverse_digits(x); }
```

Same peel-a-digit pattern as the recursive digit sum (Section 4), built iteratively. Mentioning the overflow case unprompted (2147483647 reversed doesn't fit) is what separates a strong answer.

Section 9 — More Output-Tracing & Memory Traps

Q75. What is the output? (String literal vs array — crashes are a feature here)

```
#include <stdio.h>
int main(void)
{
    char a[] = "hello";    /* array: literal COPIED to stack */
    char *p = "hello";     /* pointer to read-only literal */
    a[0] = 'H';
    printf("%s\n", a);
    p[0] = 'H';            /* ??? */
    printf("%s\n", p);
    return 0;
}
```

Solution

Prints **Hello**, then **crashes (segmentation fault)** — or formally, undefined behaviour. The array `a` owns a writable copy of the string; `p` points into the read-only `.rodata` section where literals live, so writing through it traps. Also: `sizeof(a) = 6` (includes `'\0'`), `sizeof(p) = 8`. One of the most-asked pointer questions in existence.

Q76. What is the output? (sizeof does NOT evaluate its operand)

```
#include <stdio.h>
int main(void)
{
    int i = 5;
    size_t s = sizeof(i++);
    printf("%zu %d\n", s, i);

    int arr[sizeof(int) * 2];
    printf("%zu\n", sizeof(arr) / sizeof(arr[0]));
    return 0;
}
```

Solution

Output: **4 5** then **8**. `sizeof` is computed at COMPILE time from the type of the expression; the expression is never executed, so `i++` never happens and `i` stays 5. The array has `sizeof(int)*2 = 8` elements. (Exception to the rule: variable-length arrays, whose `sizeof` is runtime.)

Q77. What is the output? (switch fallthrough — and why hardware people should care)

```
#include <stdio.h>
int main(void)
{
    int state = 2;
    switch (state) {
        case 1: printf("IDLE ");
        case 2: printf("RUN ");
        case 3: printf("DONE "); break;
        case 4: printf("ERR ");
        default: printf("DEF ");
    }
    printf("\n");
    return 0;
}
```

Solution

Output: **RUN DONE**. Without `break`, control FALLS THROUGH into the next case: entry at case 2 executes case 2 and case 3, then `break` exits. Fallthrough is exactly like forgetting a `break` in a Verilog-style priority

chain — occasionally intentional (comment it!), usually a bug. Every FSM-in-C question (Q75) hinges on getting breaks right.

Q78. What is the output? (Comma operator and ternary)

```
#include <stdio.h>
int main(void)
{
    int a = (1, 2, 3);
    int b = 1, 2, 3;          /* does this compile? */
    int x = 5;
    int y = (x > 3) ? (x > 8) ? 100 : 50 : 10;
    printf("%d %d\n", a, y);
    return 0;
}
```

Solution

The line `int b = 1, 2, 3;` does NOT compile (comma acts as a declarator separator there — '2' is not a name). Removing it: `a = 3` — the comma operator evaluates left to right and yields the LAST value. `y: x>3 true → inner ternary → x>8 false → 50`. Nested ternaries associate right-to-left; a chain of ternaries is C's version of a priority mux, which is why they show up in hardware-model code.

Q79. What is the output? (do-while and loop-exit values)

```
#include <stdio.h>
int main(void)
{
    int n = 0;
    do {
        printf("%d ", n);
        n++;
    } while (n < 0);

    int i;
    for (i = 0; i < 5; i++)
        if (i == 3) break;
    printf("| i=%d\n", i);
    return 0;
}
```

Solution

Output: **0 | i=3**. A do-while always executes its body at least ONCE — condition checked at the bottom — so 0 prints even though `n<0` is immediately false. The for-loop breaks with `i` frozen at 3 (the increment doesn't run after break). do-while is the natural shape for 'send command, then poll until done' hardware sequences.

Q80. What is the difference between these two, and what is the output? (Array of pointers vs pointer to array)

```
#include <stdio.h>
int main(void)
{
    int a[2][3] = {{1,2,3},{4,5,6}};
    int *p1[3];          /* declaration A */
    int (*p2)[3] = a;     /* declaration B */

    printf("%d %d\n", p2[1][2], *((p2 + 1) + 1));
    printf("%zu %zu\n", sizeof(p1), sizeof(*p2));
    return 0;
}
```

Solution

A: `p1` is an ARRAY of 3 pointers-to-int (`sizeof` = 24 on 64-bit). B: `p2` is one POINTER to an array of 3 ints — the correct type for walking a 2-D array's rows; `p2+1` jumps a whole 12-byte row. Output line 1: `p2[1][2] = a[1][2] = 6`, and `*(*(p2+1)+1) = a[1][1] = 5`. Line 2: **24 12**. The `[]` binds tighter than `*`, hence the parentheses in

B — read declarations inside-out.

Q81. Spot the THREE memory bugs. (malloc/free discipline)

```
#include <stdlib.h>
#include <string.h>
int main(void)
{
    int *p = malloc(10 * sizeof(int));
    p[10] = 42;                /* bug 1? */
    free(p);
    p[0] = 7;                  /* bug 2? */
    free(p);                  /* bug 3? */

    char *q = malloc(100);
    q = malloc(200);          /* bug 4? */
    return 0;
}
```

Solution

(1) `p[10]` writes the 11th element of a 10-element block — heap buffer overflow. (2) use-after-free: writing through `p` after free is UB. (3) double free: corrupts the allocator, a classic security hole. (4) reassigning `q` without freeing the first 100 bytes — memory leak. Also worth saying: always check malloc's return for NULL, and note `calloc` = malloc + zero-fill, `realloc` = resize (may move the block). Firmware often bans free entirely to avoid fragmentation — good judgment point to raise.

Q82. Dynamically allocate an $r \times c$ 2-D array of ints, use it as `m[i][j]`, and free it correctly. (Your Lab 12 topic, interview version)

Solution

```
/* Method 1: array of row pointers */
int **m = malloc(r * sizeof(int *));
for (int i = 0; i < r; i++)
    m[i] = malloc(c * sizeof(int));
m[2][3] = 7;
for (int i = 0; i < r; i++) free(m[i]); /* free rows FIRST */
free(m);

/* Method 2: one contiguous block (cache/DMA friendly) */
int *flat = malloc(r * c * sizeof(int));
#define M(i,j) flat[(i)*c + (j)]
M(2,3) = 7;
free(flat);
```

Method 1 gives real `m[i][j]` syntax but scatters rows across the heap ($r+1$ allocations, $r+1$ frees — order matters: rows before the spine). Method 2 is one allocation, contiguous — the layout hardware DMA and caches want — with manual row-major indexing $i*c+j$, the same flattening as Q32's 2-D address formula.

Q83. What is the output? (static local variable — state that persists like a register)

```
#include <stdio.h>
void tick(void)
{
    static int count = 0; /* initialized ONCE */
    int temp = 0;
    count++; temp++;
    printf("%d%d ", count, temp);
}
int main(void)
{
    tick(); tick(); tick();
    return 0;
}
```

Solution

Output: **11 21 31**. count lives in static storage and keeps its value across calls (initialized once, before main); temp is re-created on the stack each call. A static local is the closest C analog of a flip-flop inside a module: private state that survives between 'clock ticks' (calls).

Section 10 — Classic Embedded Structures: FIFO, FSM, Function Pointers

Q84. Implement a fixed-size circular FIFO (ring buffer) of bytes with put/get, distinguishing FULL from EMPTY. This is a near-guaranteed question for hardware-adjacent roles.

Solution

```
#define N 8                                /* power of 2 -> cheap wrap with mask */

typedef struct {
    unsigned char buf[N];
    unsigned head;           /* write index */
    unsigned tail;          /* read index */
} fifo_t;

int fifo_empty(fifo_t *f) { return f->head == f->tail; }
int fifo_full (fifo_t *f) { return ((f->head + 1) & (N-1)) == f->tail; }

int fifo_put(fifo_t *f, unsigned char v)
{
    if (fifo_full(f)) return -1;
    f->buf[f->head] = v;
    f->head = (f->head + 1) & (N - 1);
    return 0;
}

int fifo_get(fifo_t *f, unsigned char *v)
{
    if (fifo_empty(f)) return -1;
    *v = f->buf[f->tail];
    f->tail = (f->tail + 1) & (N - 1);
    return 0;
}
```

Design decisions you MUST be able to defend: (1) empty = head==tail; full = head+1==tail, sacrificing one slot to disambiguate — the alternative is a count field or an extra pointer bit, exactly the extra-MSB trick used in hardware FIFOs. (2) power-of-2 size lets wrap be a mask instead of %. (3) If an ISR is the producer and main the consumer, head and tail must be volatile — and single-producer/single-consumer with one writer per index needs no lock. Follow-up they WILL ask: 'how do you cross clock domains?' → Gray-coded pointers (Section 8 Q1).

Q85. Implement a simple FSM in C for a sequence detector that outputs 1 when it sees the serial pattern 1-0-1 (overlapping allowed). Use an enum + switch.

Solution

```
typedef enum { S_IDLE, S_1, S_10 } state_t;

int fsm_step(state_t *s, int in)          /* one call = one clock */
{
    int out = 0;
    switch (*s) {
        case S_IDLE: *s = in ? S_1 : S_IDLE;          break;
        case S_1:    *s = in ? S_1 : S_10;            break;
        case S_10:   if (in) { out = 1; *s = S_1; }    /* overlap: reuse the 1 */
                     else { *s = S_IDLE; }            break;
        default:     *s = S_IDLE;                      break;
    }
    return out;
}
```

```
}
```

Direct C mirror of a Moore/Mealy FSM: enum = state encoding, switch = next-state logic, the default arm = recovery from illegal states (same reason you cover default in RTL case statements). Overlapping detection means after a hit on ...101, the trailing 1 seeds the next match (go to S_1, not S_IDLE) — the detail interviewers check.

Q86. Declare a function pointer, an array of function pointers, and show how an interrupt vector table / command dispatcher uses them.

Solution

```
void uart_isr(void); void timer_isr(void); void dma_isr(void);

void (*fp)(void) = uart_isr;          /* pointer to void(void) function */
fp();                                 /* call through the pointer */

/* Vector table: index by IRQ number, jump to handler */
void (*vector_table[])(void) = { uart_isr, timer_isr, dma_isr };
vector_table[1]();                    /* dispatch timer_isr */

/* Callback registration, the driver-API pattern: */
typedef void (*cb_t)(int status);
static cb_t done_cb;
void register_cb(cb_t f) { done_cb = f; }
/* later, from the ISR: */ if (done_cb) done_cb(0);
```

The actual hardware vector table in Cortex-M IS an array of function pointers in flash — the NVIC fetches the address and jumps. Reading the declaration: `void (*vt[])(void) = vt` is an array of pointers to functions taking void returning void (parenthesize the * or you declare a function returning a pointer). Callbacks decouple drivers from applications — the C ancestor of every event-driven framework.

Q87. Compute an 8-bit checksum (two's-complement style) over a buffer such that summing the buffer PLUS the checksum byte yields 0. Then explain in one line how CRC differs.

Solution

```
unsigned char checksum(const unsigned char *p, int n)
{
    unsigned char sum = 0;
    while (n--) sum += *p++;          /* mod-256 sum */
    return (unsigned char)(0 - sum);   /* two's complement */
}
/* receiver: sum over data + checksum byte == 0 -> intact */
```

Appending the negated sum makes the total wrap to zero — the scheme used by Intel HEX records and many simple sensor protocols. CRC differs fundamentally: it's polynomial division over GF(2) implemented with XORs and shifts (an LFSR in hardware), catching burst errors that additive checksums miss. If they follow up, sketch the LFSR: shift in a bit, XOR the polynomial taps when the MSB is 1.

Q88. TRICKY — Debug round. This driver snippet compiles but misbehaves. Find all FIVE problems.

```
#define STATUS (*(unsigned int *)0x40021004) /* line 1 */
#define TIMEOUT 100000;                     /* line 2 */

int wait_ready(void)
{
    int i;
    for (i = 0; i < TIMEOUT; i++);          /* line 3 */
    {
        if (STATUS & 0x1 == 0x1)           /* line 4 */
            return 1;
    }
    return 0;                               /* line 5 */
```

```
}
```

Solution

(1) STATUS lacks **volatile** — the optimizer may read it once or delete reads. (2) Trailing **semicolon in the #define** — expands into the for condition as `i < 100000;;`, a syntax error, or corrupts other uses. (3) Stray **semicolon after for(...)** — the loop body is empty and the block below runs once, after the loop. (4) Precedence: `0x1 == 0x1` evaluates first (`=1`), so this is `STATUS & 1` — accidentally 'works' here but is wrong in general; needs `(STATUS & 0x1) == 0x1`. (5) Logic: the loop burns all iterations then checks ONCE — the status test belongs INSIDE the loop with an early return, otherwise it's a delay, not a timeout poll. Debug-the-snippet questions like this are increasingly common because they test everything at once.

Q89. Write bit-banded code to transmit a byte MSB-first on a GPIO data line with a clock line (software SPI flavor).

Solution

```
#define GPIO_OUT (*(volatile unsigned int *)0x40020014u)
#define CLK_PIN  (1u << 0)
#define DAT_PIN  (1u << 1)

void send_byte(unsigned char b)
{
    for (int i = 7; i >= 0; i--) {          /* MSB first */
        if ((b >> i) & 1) GPIO_OUT |=  DAT_PIN;
        else               GPIO_OUT &= ~DAT_PIN;
        GPIO_OUT |=  CLK_PIN;              /* clock high: slave samples */
        GPIO_OUT &= ~CLK_PIN;             /* clock low */
    }
}
```

Combines nearly everything in this set: volatile register access, set/clear masks, and bit extraction with a downward index for MSB-first order (LSB-first would loop 0..7). Real code adds delays for timing and reads MISO between edges — mention that. This is a favorite 'write real code on the whiteboard' finisher.

Q90. What is the output? (Everything-at-once trace: statics, shifts, pointers)

```
#include <stdio.h>
unsigned step(void)
{
    static unsigned lfsr = 0xACE1u;          /* 16-bit seed */
    unsigned bit = ((lfsr >> 0) ^ (lfsr >> 2) ^
                    (lfsr >> 3) ^ (lfsr >> 5)) & 1u;
    lfsr = (lfsr >> 1) | (bit << 15);
    return lfsr;
}

int main(void)
{
    printf("%04X\n", step());
    printf("%04X\n", step());
    return 0;
}
```

Solution

This is a 16-bit Fibonacci LFSR (taps 16,14,13,11 — the classic maximal-length polynomial), the standard pseudo-random generator in BIST and scramblers. Trace call 1: `lfsr = 0xACE1 = 1010 1100 1110 0001`; tap bits (positions 0,2,3,5) = 1,0,0,1 → XOR = 0; shift right and insert 0 at bit 15 → 0x5670... let's verify: `0xACE1 >> 1 = 0x5670`, `bit<<15 = 0` → **5670**. Call 2: taps of 0x5670 (0101 0110 0111 0000): bits 0,2,3,5 = 0,0,0,1 → XOR = 1 → `(0x5670>>1)|0x8000 = 0x2B38|0x8000 = AB38`. If you traced both correctly by hand, your bit-level fluency is interview-ready — and you can now explain how PRBS generators work as a bonus.

Section 11 — Pointers Deep Dive (extended)

Pointers are where hardware interviews separate candidates, because a pointer IS an address and hardware people are expected to think in addresses. Endianness detection appeared in Section 2 (Q16–Q17); this section pushes it further and then covers generic pointers, aliasing, double pointers, complex declarations and linked structures.

Q91. Print the individual bytes of the integer 0x12345678 in memory order, and state the exact output on a little-endian and on a big-endian machine.

Solution

```
#include <stdio.h>
int main(void)
{
    unsigned int x = 0x12345678;
    unsigned char *p = (unsigned char *)&x;
    for (int i = 0; i < 4; i++)
        printf("%p : %02X\n", (void *) (p + i), p[i]);
    return 0;
}
```

Little-endian (x86, most ARM): **78 56 34 12** — least-significant byte at the LOWEST address. Big-endian (network order, SPARC, some DSPs): **12 34 56 78**. The pointer never changes; only the interpretation of which byte lives where does. Note the cast MUST be to `unsigned char *` — char pointers are explicitly allowed to alias any object, which is why this trick is legal C.

Q92. Detect endianness THREE different ways: char pointer, union, and without dereferencing any int (pointer-arithmetic style). Then write one function that is endianness-INDEPENDENT.

Solution

```
/* 1. char pointer */
int le1(void){ unsigned int x = 1; return *(char *)&x == 1; }

/* 2. union */
int le2(void){ union { unsigned int i; unsigned char c; } u = {1}; return u.c == 1; }

/* 3. short/char pair, no int deref */
int le3(void){ unsigned short s = 0x0102;
               unsigned char *p = (unsigned char *)&s;
               return p[0] == 0x02; }

/* 4. ENDIANNESS-INDEPENDENT byte assembly: works everywhere */
unsigned int load_be32(const unsigned char *b) /* read big-endian bytes */
{
    return ((unsigned)b[0] << 24) | ((unsigned)b[1] << 16) |
           ((unsigned)b[2] << 8) | (unsigned)b[3];
}
```

The first three DETECT the host order; the fourth avoids caring about it. Key interview insight: production code that parses a protocol should never branch on host endianness — it should assemble values from explicit byte indices with shifts, as in `load_be32`. Shifts operate on VALUES, not memory layout, so that function gives the same result on any machine.

Q93. What is the output? Explain why a struct pointer can find a member's address without dereferencing anything.

```
#include <stdio.h>
typedef struct { unsigned int CTRL; unsigned int STATUS; unsigned int DATA; } reg_t;
int main(void)
{
    reg_t *r = (reg_t *)0x40020000u;
    printf("%p\n", (void *)&r->DATA);
    printf("%p\n", (void *) (r + 1));
    printf("%zu\n", (size_t)&(((reg_t *)0)->STATUS));
    return 0;
}
```

```
}
```

Solution

Line 1: **0x40020008** — DATA is the third 4-byte member, so base + 8. Line 2: **0x4002000c** — r+1 advances by one WHOLE struct (12 bytes). Line 3: **4** — the offsetof trick from Section 6: casting 0 to reg_t* and taking &member is pure address arithmetic, no memory is ever read. This is why you can compute register addresses at compile time. Nothing here dereferences the hardware, so the code is safe even if the peripheral is powered down.

Q94. Write a GENERIC swap function that works for any data type using void pointers.

Solution

```
#include <string.h>
void swap(void *a, void *b, size_t size)
{
    unsigned char *p = a, *q = b, t;
    for (size_t i = 0; i < size; i++) {
        t = p[i]; p[i] = q[i]; q[i] = t;    /* byte-wise exchange */
    }
}
/* usage */
int x = 1, y = 2;          swap(&x, &y, sizeof x);
double d1 = 1.5, d2 = 2.5; swap(&d1, &d2, sizeof d1);
```

void* is C's generic pointer: it can hold any object address but cannot be dereferenced or arithmetic'd directly (gcc allows void* arithmetic as an extension; standard C does not) — so you cast to unsigned char* and work byte by byte. This is exactly how qsort/bsearch/memcpy achieve genericity in C. Follow-up they may ask: 'why pass size?' — because void* carries no type information at runtime.

Q95. What is the difference between these three broken pointers: NULL pointer, wild (uninitialized) pointer, and dangling pointer? Give code for each and say which is 'safest'.

Solution

```
int *a = NULL;           /* NULL:    points nowhere, on purpose */
int *b;                  /* WILD:    holds garbage */
int *c = malloc(4); free(c); /* DANGLING: points to freed memory */
```

Dereferencing all three is undefined behaviour, but NULL is the 'safest' because on most systems address 0 is unmapped, so the fault is immediate, loud and reproducible — a deterministic segfault you can debug. Wild and dangling pointers may silently corrupt live data and crash much later somewhere unrelated. Best practice: initialize every pointer (to NULL if nothing else), and set the pointer to NULL immediately after free so later use faults instead of corrupting.

Q96. Why does this function fail to allocate memory for the caller, and how do you fix it? (Double-pointer classic.)

```
void alloc_bad(int *p, int n) { p = malloc(n * sizeof(int)); }
int main(void)
{
    int *buf = NULL;
    alloc_bad(buf, 10);
    buf[0] = 1;          /* CRASH */
}
```

Solution

```
void alloc_ok(int **p, int n) { *p = malloc(n * sizeof(int)); }
alloc_ok(&buf, 10);          /* now buf is updated */

/* Alternative: return the pointer instead */
int *alloc_ret(int n) { return malloc(n * sizeof(int)); }
buf = alloc_ret(10);
```

C is ALWAYS pass-by-value — `alloc_bad` receives a copy of the pointer, so assigning to `p` changes only the local copy and `buf` stays NULL. To modify the caller's pointer you must pass its address (`int**`), just as modifying a caller's `int` requires `int*`. Rule: 'to change a T in the caller, pass a T*' — apply it with `T = int*` and you get `int**`.

Q97. Decode these declarations in plain English (the classic right-left rule exercise).

```
int *p[10];           /* A */
int (*p)[10];         /* B */
int *f(void);         /* C */
int (*f)(void);       /* D */
int (*fa[5])(int);    /* E */
int *(*f)(int *);     /* F */
```

Solution

A: array of 10 pointers to `int`. B: pointer to an array of 10 `ints`. C: function taking `void`, returning pointer to `int`. D: pointer to a function taking `void`, returning `int`. E: array of 5 pointers to functions taking `int`, returning `int` (an interrupt vector / dispatch table). F: pointer to a function taking `int*`, returning `int*`. Method: start at the identifier, go RIGHT while you can (`[]` and `()`), then LEFT (`*` and types), and parentheses override precedence. Practice these out loud — interviewers ask you to read them aloud.

Q98. What is the output? Explain why an array name is NOT a pointer variable, even though it decays to one.

```
#include <stdio.h>
int main(void)
{
    int a[5] = {1,2,3,4,5};
    int *p = a;
    printf("%zu %zu\n", sizeof(a), sizeof(p));
    printf("%d %d\n", *(a + 2), *(p + 2));
    p++;
    /* legal */
    /* a++; */ /* ERROR */
    printf("%d\n", *p);
    printf("%d %d\n", a[1], 1[a]); /* ?! */
    return 0;
}
```

Solution

Output: **20 8, 3 3, 2, 2 2**. The array name is a non-modifiable lvalue naming 20 bytes of storage — it DECAYS to `&a[0]` in most expressions but is not itself a variable holding an address, so `a++` is illegal while `p++` is fine, and `sizeof` sees the array not a pointer. Bonus: `a[1]` is defined as `*(a+1)`, and addition commutes, so `1[a]` compiles and gives the same value — a party trick that also proves indexing is just pointer arithmetic.

Q99. Given `char *argv[]` (an array of string pointers), print every argument two different ways, and explain the memory layout.

Solution

```
int main(int argc, char *argv[]) /* == char **argv */
{
    for (int i = 0; i < argc; i++) printf("%s\n", argv[i]);
    for (char **p = argv; *p != NULL; p++) printf("%s\n", *p);
    return 0;
}
```

Layout: `argv` points to a contiguous array of `char*` slots; each slot points to a separate NUL-terminated string elsewhere in memory; the slot after the last argument is NULL, which is why the second loop can stop without `argc`. Two levels of indirection: `argv` = address of the slot array, `*p` = address of one string, `**p` = the first character. This is the same shape as the Braille string table in your Lab 11.

Q100. Write `strcpy` and `strcmp` using only pointers (no array indexing). Then explain why the compact one-liner `strcpy` is dangerous.

Solution

```
char *my_strcpy(char *d, const char *s)
{
    char *ret = d;
    while ((*d++ = *s++) != '\0') ;    /* copies, then tests, incl. '\0' */
    return ret;
}

int my_strcmp(const char *a, const char *b)
{
    while (*a && (*a == *b)) { a++; b++; }
    return (unsigned char)*a - (unsigned char)*b;    /* sign of difference */
}
```

The strcpy loop copies the terminator too, because the assignment's VALUE is the character copied and '\0' is falsy — elegant, and the canonical whiteboard answer. Danger: it performs no bounds checking, so if the destination buffer is smaller than the source you get a buffer overflow — the #1 historical security bug. Say you'd use strncpy/snprintf (or better, a length-carrying API) in production. Note the unsigned char casts in strcmp: comparing signed chars gives wrong ordering for bytes above 127.

Q101. Reverse a string in place using two pointers, and reverse the WORDS of a sentence in place. (Two-pointer technique.)

Solution

```
void rev_range(char *b, char *e)    /* e points at LAST char */
{
    while (b < e) { char t = *b; *b++ = *e; *e-- = t; }
}

void rev_words(char *s)
{
    char *start = s, *p = s;
    rev_range(s, s + strlen(s) - 1);    /* 1. reverse everything */
    while (*p) {                        /* 2. re-reverse each word */
        if (*p == ' ') { rev_range(start, p - 1); start = p + 1; }
        p++;
    }
    rev_range(start, p - 1);            /* last word */
}
```

"the sky is blue" → reverse all → "eulb si yks eht" → reverse each word → "blue is sky the". The two-pointer converge-from-both-ends pattern is the same one used in the byte-buffer reversal (Section 5) and in palindrome checks. Doing it in place with O(1) extra space is what the interviewer is looking for.

Q102. Detect whether a singly linked list has a cycle, and find the middle node in ONE pass. (Floyd's two-pointer / slow-fast technique.)

Solution

```
typedef struct node { int val; struct node *next; } node_t;

int has_cycle(node_t *head)
{
    node_t *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;    /* 1 step */
        fast = fast->next->next;    /* 2 steps */
        if (slow == fast) return 1;    /* they meet inside the loop */
    }
    return 0;    /* fast hit NULL: no cycle */
}

node_t *middle(node_t *head)
{
    node_t *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
```

```

    return slow;                                /* slow is at the midpoint */
}

```

If there is a cycle, the fast pointer laps the slow one and they must collide (relative speed 1 node/step). If there isn't, fast reaches NULL. The same loop structure lands slow at the middle when fast reaches the end. Always guard both fast and fast->next before the double step, or you dereference NULL on even-length lists.

Q103. Reverse a singly linked list iteratively. (The most-asked pointer-manipulation question in existence.)

Solution

```

node_t *reverse(node_t *head)
{
    node_t *prev = NULL, *cur = head, *next = NULL;
    while (cur) {
        next = cur->next;    /* 1. save the rest of the list */
        cur->next = prev;    /* 2. flip this link backwards */
        prev = cur;         /* 3. advance prev */
        cur = next;         /* 4. advance cur */
    }
    return prev;           /* prev is the new head */
}

```

Three pointers, four steps, in that exact order — saving next BEFORE overwriting cur->next is the whole trick; swap the order and you lose the rest of the list. Draw the boxes-and-arrows diagram on the whiteboard as you go; interviewers grade the explanation as much as the code.

Q104. What is pointer aliasing, why does it block optimization, and what does restrict do?

```

void add(int *a, int *b, int *sum, int n)
{
    for (int i = 0; i < n; i++) sum[i] = a[i] + b[i];
}

```

Solution

Aliasing = two pointers may refer to overlapping memory. The compiler cannot prove sum doesn't overlap a or b, so it must RELOAD a[i] and b[i] every iteration (a store through sum might have changed them) — blocking vectorization and register caching. Adding restrict (int * restrict sum) is your promise that this object is accessed only through that pointer, letting the optimizer keep values in registers. Break the promise and you get UB. Related rule: strict aliasing says you may not access an object through a pointer of an incompatible type — except via char*, which is why byte-wise inspection (Q79) is legal.

Q105. How do you check whether a pointer is properly aligned for its type, and why does it matter on hardware?

Solution

```

int is_aligned(const void *p, size_t align)    /* align must be 2^k */
{
    return ((uintptr_t)p & (align - 1)) == 0;
}
/* usage */
if (!is_aligned(buf, 4)) { /* cannot safely cast to unsigned int */ }

```

Cast to uintptr_t (the integer type guaranteed to hold a pointer, from stdint.h) and test the low log2(align) bits — the same masking as the align-up trick in Section 1. Why it matters: many CPUs fault or silently misbehave on unaligned word accesses, and DMA engines and bus bridges usually REQUIRE naturally aligned transfers. This is also why packed structs are risky: a member may land unaligned, so taking its address and dereferencing can trap.

Q106. What is the output? (Pointer difference, comparison, and the const trap)

```

#include <stdio.h>
int main(void)
{
    int a[5] = {10,20,30,40,50};
    int *p = &a[1], *q = &a[4];
    printf("%td %td\n", q - p, (char *)q - (char *)p);
    printf("%d\n", p < q);

    const int v = 5;
    const int *cp = &v;
    /* *cp = 6; */           /* compile error */
    int *bad = (int *)cp;    /* casting away const */
    *bad = 6;                /* legal syntax, UB at runtime */
    printf("%d\n", v);
    return 0;
}

```

Solution

Line 1: **3 12** — pointer subtraction yields the number of ELEMENTS between them (3 ints), while casting to char* gives raw bytes (12). Line 2: **1** — pointers into the same array are comparable. Line 3: **undefined** — casting away const and writing to a genuinely const object is UB; the compiler may have placed v in read-only memory or constant-folded the printf to print 5 even though memory says 6. Only cast away const when the object was not originally const (e.g. an API that forgot const).

Q107. Implement a generic memset and a generic array-search using void* and a comparison callback. (Combines void*, function pointers and byte access.)

Solution

```

void *my_memset(void *s, int c, size_t n)
{
    unsigned char *p = s;
    while (n--) *p++ = (unsigned char)c;    /* c truncated to a byte */
    return s;
}

void *find(const void *base, size_t count, size_t size,
           const void *key, int (*cmp)(const void *, const void *))
{
    const unsigned char *p = base;
    for (size_t i = 0; i < count; i++) {
        const void *elem = p + i * size;    /* manual indexing */
        if (cmp(elem, key) == 0) return (void *)elem;
    }
    return NULL;
}

```

Two ideas hardware C leans on constantly: (1) memset takes an INT but writes bytes — memset(buf, 1, n) fills 0x01010101, not the value 1 per int, a very common bug when clearing int arrays (only 0 works as expected). (2) Because void* has no type, you must be handed the element size and step manually — p + i * size — which is exactly how qsort works.

Q108. TRICKY — What is the output? (Pointer to pointer with arrays of strings, from your practice.c)

```

#include <stdio.h>
void test(char **ptr)
{
    char *ptr1 = (ptr + 2)[-1];    /* ?? */
    printf("%s\n", ptr1);
    printf("%c\n", *((ptr + 1) + 1));
    printf("%s\n", *ptr + 1);
}
int main(void)
{
    char *arr[] = {"ant", "bat", "cat", "dog"};
}

```

```

    test(arr);
    return 0;
}

```

Solution

Line 1: `(ptr+2)[-1] = *(ptr+2-1) = ptr[1] = bat`. Negative indices are legal as long as the resulting address is in range — indexing is just pointer arithmetic. Line 2: `*(ptr+1)` is the string "bat"; +1 then dereference gives its second character = **a**. Line 3: `*ptr` is "ant", and `*ptr + 1` advances the CHAR pointer one position, printing **nt**. Watch the difference between `*(p+1)` (next string) and `*p + 1` (next character of the first string) — that single space is the whole question.

Q109. A register at 0x40000000 is 32 bits wide. Write code to read its byte 2 (bits [23:16]) via a pointer AND via shifting. Which should you use on real hardware, and why?

Solution

```

#define REG (*(volatile unsigned int *)0x40000000u)

/* Method A: byte pointer */
volatile unsigned char *bp = (volatile unsigned char *)0x40000000u;
unsigned char b2_ptr = bp[2];          /* little-endian: byte 2 = bits [23:16] */

/* Method B: shift and mask */
unsigned char b2_shift = (REG >> 16) & 0xFFu;

```

Use **Method B**. Two reasons: (1) Method A is endianness-dependent — on a big-endian machine `bp[2]` is bits [15:8]. (2) More importantly, many peripherals only support 32-bit bus accesses; an 8-bit read may return garbage, fault, or trigger the wrong side effect (some status registers clear-on-read, and a narrow access can clear the wrong lane). Shifting operates on the VALUE after a legal 32-bit read, so it is portable and safe. This 'why not just point at the byte?' question is a great one to get right.

Q110. Final pointer boss — trace the complete output on a 64-bit little-endian machine.

```

#include <stdio.h>
int main(void)
{
    unsigned int a[3] = {0x11223344, 0x55667788, 0x99AABBCC};
    unsigned int *p = a;
    unsigned char *c = (unsigned char *)a;

    printf("%X\n", *(p + 1));
    printf("%02X\n", c[4]);
    printf("%02X\n", *((unsigned char *) (p + 2) + 3));
    printf("%td\n", (unsigned char *) (p + 2) - c);
    p += 2; c += 2;
    printf("%X %02X\n", *p, *c);
    return 0;
}

```

Solution

```

55667788
88
99
8
99AABBCC 22

```

(1) `p+1` skips one int → `a[1]`. (2) `c[4]` is the 5th BYTE = first byte of `a[1]` = 0x88 (little-endian LSB first). (3) `p+2` is `a[2]`; as a byte pointer +3 gives the HIGHEST byte of `a[2]` = 0x99. (4) Two ints = 8 bytes of separation. (5) `p+=2` moves 8 bytes to `a[2]`; `c+=2` moves only 2 bytes into `a[0]`, whose byte 2 is 0x22. Same starting address, different step sizes — if you got all five, pointer arithmetic is solid.

Appendix — C vs C++: What Changes for These Questions

About 90% of this set — every bitwise, masking, shifting, endianness, FIFO and LFSR question — is identical in C++, which inherits C's integer arithmetic and memory model. RTL/firmware roles overwhelmingly interview in C; C++ appears mainly for SystemC modeling and SystemVerilog DPI-based verification. Below are the concrete deltas, keyed to the questions they affect. Knowing these cold lets you flag them proactively — a strong signal in any interview.

Q111. APPENDIX — `sizeof('A')` differs between the languages. (Affects the Section 7 question.)

Solution

In C, a character constant has type `int`, so `sizeof('A') = 4`. In C++ it has type `char`, so `sizeof('A') = 1`. This is the single most famous C/C++ divergence and a favorite quick-fire question. (`sizeof(1.5)=8` and `sizeof(1.5f)=4` are the same in both.)

Q112. APPENDIX — Union type punning is legal C but undefined behaviour in C++. What is the C++-correct way? (Affects the endianness-union and float-bits questions.)

Solution

```
/* C (fine): */
union { float f; unsigned u; } p = { .f = 1.0f }; /* read p.u */

/* C++ correct alternatives: */
unsigned u;
std::memcpy(&u, &f, sizeof u); /* any C++ standard */

auto u2 = std::bit_cast<unsigned>(f); /* C++20, compile-checked */
```

In C++, writing one union member and reading another is formally UB (only one member is 'active'). Compilers usually tolerate it, but the expected interview answer is `memcpy` (the optimizer removes the actual copy) or C++20 `std::bit_cast`. The char-pointer endianness test remains valid in both languages because char pointers may alias anything.

Q113. APPENDIX — `malloc/free` vs `new/delete`: what changes in the memory-bug questions?

Solution

```
int *p = (int *)malloc(10 * sizeof(int)); /* C++ REQUIRES the cast */
int *q = new int[10]; /* C++ way; calls ctors */
delete[] q; /* [] MANDATORY for arrays */
delete q; /* <- UB if q was new[]: the classic C++-only bug */

std::unique_ptr<int[]> r(new int[10]); /* RAII: auto-freed */
```

Three deltas: (1) C++ forbids implicit `void*` conversion, so `malloc` needs a cast (in C, casting `malloc` is discouraged). (2) `new/delete` add a new bug class — mismatching `new[]` with `delete` — on top of the leak/double-free/use-after-free bugs from Section 9. (3) The modern-C++ expected answer to 'how do you prevent leaks?' is RAII/smart pointers, not discipline. Firmware C++ often still bans heap use after init for the same fragmentation reasons as C.

Q114. APPENDIX — String literals: how does the `char *p = "hello"` trap change?

Solution

In C, a string literal has type `char[]` and the write `p[0]='H'` compiles but crashes at runtime (read-only memory). In C++ literals are `const char[]`, so `char *p = "hello";` is ill-formed (invalid conversion) — the runtime crash becomes a compile-time error. Write `const char *p = "hello";`. Same underlying storage, stricter type system.

Q115. APPENDIX — enum vs enum class: what happens to the FSM and enum-arithmetic questions?

Solution

```
enum month { jan, feb, mar, apr = 5, may, jun }; /* C & C++ */
int s = jan + mar + may; /* = 8, both langs */

enum class state_t { IDLE, S1, S10 }; /* C++11 scoped */
int x = state_t::IDLE + 1; /* ERROR: no implicit int conversion */
int y = (int)state_t::S1 + 1; /* must cast explicitly */
```

Plain enums behave identically in both languages (the Quiz 16 arithmetic still gives 8). C++11 enum class adds scoping (`state_t::IDLE`) and removes implicit conversion to `int` — so enum arithmetic questions become compile errors unless cast. For FSMs, enum class is the modern preference precisely because it prevents accidental state/int mixing.

Q116. APPENDIX — volatile register access in C++: syntax and the C++20 caveat.

Solution

```
/* C style (works in C++ too): */
#define REG (*(volatile uint32_t *)0x40021000u)

/* Idiomatic C++: */
auto &REG2 = *reinterpret_cast<volatile uint32_t *>(0x40021000u);

REG2 = REG2 | (1u << 3); /* safe in all standards */
REG2 |= (1u << 3); /* deprecated in C++20!
```

Semantics of `volatile` are identical; C++ just prefers `reinterpret_cast` (greppable, explicit). C++20 deprecated compound assignment (`|=`, `+=`) and `++/--` on `volatile` lvalues because they hide a read-modify-write that is not atomic — write the load and store explicitly. Mentioning this shows current knowledge. (C++23 un-deprecated `|=` and `&=` for exactly the register use case, so the safest interview answer is: 'I write the RMW explicitly to make the two bus accesses visible.')

Q117. APPENDIX — Macros vs inline vs constexpr: how does the SQR(x) question upgrade?

Solution

```
#define SQR(x) ((x) * (x)) /* still evaluates x twice */
inline int sqr(int x) { return x * x; } /* C99/C++: type-safe */
constexpr int sqrc(int x) { return x * x; } /* C++11: usable at COMPILE */

int arr[sqrc(4)]; /* array size — C needs a macro
                  or enum for this */
```

The `SQR(i++)` double-evaluation bug is identical in both languages. The upgrade path differs: C++ `constexpr` functions can be used where C requires macros (array bounds, case labels), removing the last legitimate reason for function-like macros. C interviewers accept 'macro for compile-time constants'; C++ interviewers expect 'constexpr'.

Q118. APPENDIX — Why does mixing SystemVerilog DPI or C libraries with C++ need extern "C"? (The verification-role question.)

Solution

```
/* my_dpi.cpp */
extern "C" int dpi_checksum(const unsigned char *p, int n)
{
    /* callable from SystemVerilog:
       import "DPI-C" function int dpi_checksum(...); */
}
```

C++ name-mangles symbols to encode overloads (`dpi_checksum` becomes something like `_Z12dpi_checksumPKhi`), but SystemVerilog DPI, the C runtime and linkers expect the plain C name. `extern`

"C" disables mangling for that function, at the cost of losing overloading. Every mixed C/C++ codebase wraps its headers in `#ifdef __cplusplus extern "C" { #endif` for the same reason. In verification interviews (UVM testbench calling a C reference model), this is a near-certain question.

Q119. APPENDIX — Rapid-fire summary table: state each delta in one line.

Solution

- `sizeof('A')`: 4 in C, 1 in C++.
- Union punning: OK in C; use `memcpy/std::bit_cast` in C++.
- `malloc` returns `void*`: implicit conversion in C, cast required in C++.
- `new[]/delete[]` mismatch: a bug class that exists only in C++.
- String literals: `char[]` in C (runtime crash if written), `const char[]` in C++ (compile error).
- enum class (C++11): scoped, no implicit int conversion — enum arithmetic stops compiling.
- `volatile x |= ...`: fine in C, deprecated C++20 — write the read-modify-write explicitly.
- Compile-time constants: macros/enums in C, `constexpr` in C++.
- struct tag: C needs `'struct S s;'` or a typedef; C++ allows plain `'S s;'`.
- Linking across languages: `extern "C"` to defeat name mangling (DPI, drivers).
- Everything bitwise — masks, shifts, Gray code, FIFO index math, LFSRs — is 100% identical.

Suggested drill plan: Day 1 — Section 1 twice (bit ops must become reflex). Day 2 — Sections 2 & 3 (write every answer from a blank page). Day 3 — Sections 4 & 6 traces on paper, checking against gcc afterwards. Day 4 — Section 5 problems as timed 10-minute whiteboard exercises, and rehearse Section 7 answers OUT LOUD. Re-derive, don't memorize: interviewers always mutate the question (different bit position, opposite endianness, signed instead of unsigned).